

第一阶段面试问答复习手册

手机阅读版 · RAG 基础与 Agent 面试表达

学习范围	项目定位、文档处理、Embedding、Milvus、在线路由、Agent
学习方式	第一遍直接理解，第二遍模拟面试
生成日期	2026-08-18

适合手机竖屏阅读；目录与 PDF 书签可直接跳转章节。

目录

一、项目介绍必问题	3
二、文档解析与 Chunk 高频题	7
三、Embedding 与 Milvus 高频题	9
四、路由与基础 RAG 高频题	13
五、DeepSearch 高频题	15
六、ChainOfRAG 高频题	17
七、可观测性与表达边界	19
八、面试前十分钟速记	20

范围：项目定位、文档处理、Embedding、Milvus、在线路由与 Agent。用途：第一阶段面试前集中复习。问题按“项目必问 → RAG 基础 → 向量与 Milvus → Agentic RAG → 诚实边界”排列。原则：先给结论，再讲原因、实现、取舍和证据；不把后续计划说成已有能力。配套材料：第一阶段学习讲义·项目问题与优化清单 状态更新（2026-08-18）：基础题保留，当前能力以第三阶段材料为准；不要再说项目只支持 PDF、只到文件级引用或没有产品 RBAC。

一、项目介绍必问题

1. 请介绍一下 DeepSearcher 项目

一句话回答：

DeepSearcher 是一个基于 RAG 的文档问答项目，让大模型先从用户知识库检索证据，再根据证据回答问题。

深入回答：

用户上传资料后，系统先经受控 Loader 解析或 OCR、切成 Chunk、生成 Embedding，并把向量、文本和来源写入 Milvus。用户提问时，系统选择查询策略和 Collection，将问题向量化后检索相关片段，再交给当前配置的 LLM 生成答案。当前工作台还提供 Workspace/KB 检索前授权、操作审计、消息反馈和本地目录同步；我可以从源码、测试与验证记录说明这些链路。

面试官可能追问：为什么用 RAG？你具体做了什么？项目最大的不足是什么？

诚实边界：这是基于开源项目完成的学习与改造，不能把原作者全部能力说成个人原创；当前也没有完整生产评测和高并发验证。

2. 为什么使用 RAG，而不是直接让大模型回答？

一句话回答：

RAG 能把私有、可更新的文档作为外部证据提供给模型，减少只依赖模型参数记忆造成的知识缺失和无依据回答。

深入回答：

直接问模型时，它不了解刚上传的内部文档，也难以稳定给出来源。RAG 先检索相关片段，再把片段和问题一起交给模型；知识更新时重新入库即可，不必重新训练模型。但 RAG 不能自动消除幻觉：解析、切分、检索和生成任一环节出错，最终答案仍可能错误。

诚实边界：仓库已有版本化检索/回答金标、Citation Span、Trust Consistency、Risk 和 Provenance 门禁，但这些不能外推为所有业务语料、所有模型或生产容量下的质量保证。

3.

项目的离线入库和在线查询链路是什么？

标准回答：

离线：文件 → Loader 解析/OCR → splitter 切 Chunk → Embedding → Milvus

在线：问题 → RAGRouter 选策略 → CollectionRouter 选知识库

→ 问题 Embedding → Milvus 检索 → LLM 生成答案

Loader 只负责把文件转成文档文本；Chunk 才是向量化和检索单元。在线查询也不是直接问模型，而是先路由、检索，再生成。

4. 你在项目中遇到过什么实际问题？

标准回答：

本地启动 Milvus 时遇到 Docker/WSL 内存不足，RocksDB 报 `Cannot allocate memory`。我先检查容器状态和日志，再定位到资源限制；调整内存后重新验证 Collection 写入和查询。故障实验还发现：后端离线会返回 503，而部分 Milvus 异常可能在适配层变成空列表，容易被误判为知识库无命中。

取舍与改进：生产上应区分依赖不可用、Collection 不存在、维度错误、真实无命中和低相关命中，并增加指标、告警和受控重试。

5. 项目目前最大的不足是什么？

标准回答：

当前主要不足是：第二个协作数据源、连接器 ACL 撤销同步、分布式流量治理、外部身份提供商、压测和正式 SLO 尚未完成；复杂 Agent 的成本与模型质量也仍需按运行配置持续评测。

我会优先在目标业务资料、模型和部署配置上复跑现有金标与质量门禁，分别检查 Recall@K、答案正确率、引用正确率、延迟和 Token。没有与目标场景匹配的评测，就无法证明 Rerank、混合检索或参数调整真正有效。

二、文档解析与 Chunk 高频题

6. 为什么不能把整篇 PDF 直接生成一个向量？

一句话回答：

整篇文档只有一个向量时检索颗粒度太粗，细节问题会带回大量无关内容，也会浪费大模型上下文。

深入回答：

RAG 要找与问题最相关的局部证据。当前项目先提取文档文本，再切成 Chunk；Chunk 是实际的向量化和检索单元。代价是 Chunk 太小会割裂语义，所以项目使用 overlap 缓解边界断裂，并用 `wider_text` 为生成阶段补充上下文。

7. `chunk_size` 和 `chunk_overlap` 应该如何确定？

一句话回答：

它们要在检索颗粒度、上下文完整性、重复率和成本之间取舍，并通过评测选择，不能只靠经验。

深入回答：

Chunk 太大时语义完整但检索粗、单片段 Token 高；太小时检索更细，但语义容易断裂，向量数量增加。overlap 太小无法缓解边界，太大会制造重复向量和重复命中。当前 1500/100 只是默认值；实测改成 500/50 后 Chunk 从 10 增加到 25，但这不能证明哪组更好。

验证方法：准备带标准证据片段的问题集，比较 Recall@K、回答质量、重复率、延迟、存储和 Token。

8. overlap 和 wider_text 有什么区别？

标准回答：

overlap 发生在切分阶段，让相邻 Chunk 重复少量文字，解决切分边界断裂；wider_text 是命中 Chunk 周围的更宽上下文，主要在回答生成阶段使用。项目采用“短文本检索、宽上下文生成”的思路。

9. 当前 PDF 解析有什么限制？

标准回答：

PDF 解析器主要处理文本层，适合可复制文字和简单排版；扫描件会走 OCR，但多栏和复杂版面仍可能破坏阅读顺序。当前系统已支持 Excel、PPTX、图片 OCR 与 Citation/Evidence 内部定位，PDF 也有页码/位置相关验证；但这不是视觉版面理解或“引用即语义证明”，仍需报告解析与 Trust 校验状态。

三、Embedding 与 Milvus 高频题

10. Embedding

是什么？它能保证答案正确吗？

标准回答：

Embedding 把文本映射为固定维度的浮点向量，使语义相近的文本在向量空间中更接近。项目对文档 Chunk 和用户问题分别生成向量，再由 Milvus 寻找近邻。它只负责语义表示和候选召回，不能保证事实正确、正确片段一定召回或最终回答没有幻觉。

11. 为什么向量维度必须与 Collection 一致？

标准回答：

Milvus 创建 `FLOAT_VECTOR` 字段时已经固定维度，插入和查询向量必须满足同一 Schema，才能逐坐标计算距离。1024 维查询不能搜索 1536 维 Collection。

继续追问：两个模型都是 1024 维能否混用？

不能直接混用。维度相同不代表语义坐标系相同；模型版本、预处理和向量空间都必须一致。

12. L2 距离能否直接叫相似度？

标准回答：

不能笼统说“分数越大越相似”。当前默认 L2 是距离，越小越近。项目虽然使用 `score` 字段名，实际填入的是 Milvus 的 `distance`。真实实验中 0.01 比 0.81 和 4.81 更近。

13. L2、Cosine 和 IP 如何选择？

标准回答：

L2 比较绝对几何距离，Cosine 更关注方向，IP 比较内积并受模长影响。应该优先遵循 Embedding 模型建议，确认是否归一化，再用真实问题集比较 Recall@K，不能仅凭名称选择。

14. 为什么切换 Embedding 模型通常要重新入库？

标准回答：

新模型可能改变维度、语义空间、归一化和推荐 metric，旧文档向量不能与新查询向量可靠比较。更安全的做法是创建新版本 Collection，重新生成向量并回归评测，验证后切流，保留旧版本回滚。

15. 为什么选择 Milvus？和 FAISS、pgvector 如何取舍？

标准回答：

Milvus 适合把向量检索作为独立数据服务，提供持久化、索引、Collection 和客户端接口；FAISS 更适合单机算法实验，服务化和权限要自己补；pgvector 适合已经使用 PostgreSQL、向量需要与关系数据联合查询的场景。选型要看规模、过滤需求、事务需求、团队基础设施和运维成本，不能说 Milvus 性能最好。

16. Milvus Lite 和 Docker Milvus 如何选择？

标准回答：

Lite 适合个人学习、原型和小数据测试，门槛低；Docker Milvus 是独立网络服务，更适合演练连接、资源限制和服务故障。Lite 测试通过不能证明 Docker 部署正常，Docker 单机跑通也不能包装成高可用生产集群。

17. 为什么 Milvus 空结果不一定表示没有答案？

标准回答：

空结果还可能来自真实无命中、查询范围或参数不合理；Collection 不存在、连接失败、维度不匹配和搜索异常在当前 Milvus 适配层会映射为明确错误，而不是统一变成 []。生产上仍必须区分系统失败、搜索成功但零命中和低相关命中。

18. `force_new_collection=True` 有什么风险？

标准回答：

它可能删除已有 Collection 后重建，本地实验方便，但生产上可能造成向量数据丢失。应使用版本化 Collection、迁移验证、切流和回滚流程，并限制删除权限。

四、路由与基础 RAG 高频题

19. RAGRouter 和 CollectionRouter 有什么区别？

标准回答：

RAGRouter 决定“使用哪种查询策略”，CollectionRouter 决定“查询哪些知识库”。前者当前在 DeepSearch、ChainOfRAG 和 NaiveRAG 之间选择；后者只输出实际存在且当前请求允许的 Collection 名称列表。

20. 为什么使用 LLM 做路由？有什么风险？

标准回答：

LLM 能理解开放的自然语言语义，比大量关键词规则灵活；代价是输出可能不稳定、格式错误、索引越界或生成不存在的 Collection，而且会增加 Token 和延迟。生产上需要结构化输出、范围和白名单校验、安全 fallback、路由日志及评测。

项目边界：当前路由已校验候选编号、集合 allowlist 并提供 fallback；这只能避免非法输出和越权范围，不能保证模型为每个问题选到质量最优的策略或集合。

21. 为什么 NaiveRAG 仍然重要？

标准回答：

NaiveRAG 只做一次检索和一次生成，成本低、延迟稳定、容易排查，是衡量复杂 Agent 是否真正带来收益的 baseline。多轮 Agent 必须证明正确率或证据覆盖提升值得额外 Token、延迟和失败风险。

22. 如何区分检索失败和生成失败？

标准回答：

先检查标准证据是否进入 top-k。正确证据未进入是检索侧问题；证据正确完整但模型仍答错，主要检查 Prompt、上下文组织和回答忠实度。没有结果时还要先排除 Milvus、Collection、维度和 Embedding 故障。

五、DeepSearch 高频题

23. DeepSearch 的执行流程是什么？

标准回答：

DeepSearch 先把原问题拆成最多四个子查询，并发执行 Collection 路由、向量检索；随后使用 LLM 对候选 Chunk 做 YES/NO 相关性过滤，合并去重。非最后一轮时，它根据原问题、历史子查询和已有证据生成补充查询；无新查询时提前停止，达到 `max_iter` 时强制结束，最后汇总全部证据生成答案。

24. DeepSearch 为什么成本高？

标准回答：

主要原因是每个子查询、每个 Collection 的每条候选 Chunk 都可能触发一次 LLM 相关性判断，还要计算路由、反思和最终总结。调用量近似随“子查询数 × Collection 数 × 候选数”增长。

优化方向：先去重、限制候选数、批量判断、使用专用 Reranker、小模型路由、缓存、并发上限、总 Token 与总时间预算。

25. DeepSearch 的 Rerank 是真正的重排吗？

标准回答：

严格说不是完整的打分式 Rerank，而是逐条 LLM 二分类过滤：模型只返回 YES 或 NO，决定保留或丢弃，没有为全部候选生成统一分数并重新排序。

26. 为什么 `max_iter=1` 时没有反思？

标准回答：

每轮检索后，代码先判断当前是否为最后一轮；如果是就直接退出，只有非最后一轮才生成补充查询。因此 `max_iter=1` 时反思分支根本没有执行，不是模型返回空结果。

27. DeepSearch 当前有哪些诚实边界？

标准回答：

内部仍有同步模型和数据库调用，不能声称全链路真正异步；模型输出虽有类型、范围、去重和 fallback 校验，但仍可能作出低质量策略选择；联网搜索已以可选 Tavily Provider 实现，只有显式请求且具备凭据时启用；固定评测也不能证明它在所有场景优于 NaiveRAG。

六、ChainOfRAG 高频题

28. ChainOfRAG 的执行流程是什么？

标准回答：

ChainOfRAG 每轮根据原问题和历史中间问答生成一个简单追问，再选择 Collection、向量检索并生成中间答案；随后让 LLM 选择支持该问答对的文档。中间问答进入下一轮，形成串行事实链；可选 early stopping 判断信息是否足够，最后综合支持文档和中间问答生成答案。

29. ChainOfRAG 和 DeepSearch 有什么区别？

标准回答：

DeepSearch 更像把多个研究方向并行展开，适合报告和多方面问题；ChainOfRAG 每轮只生成一个追问，下一轮依赖上一轮中间答案，更适合多跳事实链。前者主要风险是调用量大，后者主要风险是错误中间答案向后传播。

30. 为什么关闭 early stopping 后没有反思结果？

标准回答：

充分性判断被写在 `if early_stopping` 分支中。当前配置值是 `true`，会在满足最小证据条件后执行判断；显式关闭时才不会调用反思，也不会记录反思 Trace，而是运行到 `max_iter`。这不是反思失败，而是代码没有执行该分支。

31. 支持文档筛选有什么风险？

标准回答：

模型需要返回支持中间问答的文档索引列表，格式和索引都可能非法。当前校验已拒绝负数、浮点数、布尔值、重复值和越界值，并只保留满足 `0 <= index < len(results)` 的整数；未来仍可用 JSON Schema 等方式进一步收紧模型输出契约。

32. 项目中的 ChainOfRAG 是否完整复现了 CoRAG 论文？

标准回答：

没有。两者都采用动态查询改写和逐步检索思想，但当前项目是基于通用 LLM Prompt 的应用层循环，没有实现论文中的模型训练、拒绝采样中间检索链和完整测试时解码策略。准确说法是“受 CoRAG 思想启发”。

七、可观测性与表达边界

33. Trace 是不是模型思维链？

标准回答：

不是。Trace 是应用主动记录的 Agent 选择、Collection、检索文档、迭代、延迟和 Token 等执行事件，不是模型内部不可见的推理过程。准确称呼是执行 Trace 或阶段日志。

34. 如果流量扩大十倍，你会怎样改？

标准回答：

先测量 QPS、P95/P99、错误率、Token、模型调用数、Milvus 连接和资源使用，确认瓶颈所在。随后按层改造：限制复杂 Agent 预算，增加并发上限、超时和限流；复用 Embedding、批量入库、调整索引和连接；拆分入库与查询；再根据容量决定缓存、副本或集群。每一步都要回归 Recall@K，避免只优化延迟却损害召回。

诚实边界：当前没有十倍流量压测，这些是系统设计方案，不是已验证成果。

八、面试前十分钟速记

1. 项目名 DeepSearcher, 查询策略名 DeepSearch。
2. RAG 的价值是私有知识、可更新和证据, 不是彻底消除幻觉。
3. Chunk 是检索单元; overlap 与 `wider_text` 作用不同。
4. `1500/100` 没有评测证明最优。
5. 相同维度不代表不同 Embedding 模型兼容。
6. L2 越小越近, 当前 `score` 实际是 distance。
7. 空结果可能是系统故障。
8. 默认查询由 RAGRouter 在 DeepSearch、ChainOfRAG 和 NaiveRAG 中选择; NaiveRAG 是默认 fallback。
9. LLM 路由必须校验和 fallback。
10. NaiveRAG 是复杂 Agent 的成本与效果基线。
11. DeepSearch 的 Rerank 是逐条 YES/NO 过滤。
12. 当前配置启用 ChainOfRAG early stopping; 关闭它时才不产生充分性反思。
13. Trace 不是思维链。
14. ChainOfRAG 不是 CoRAG 论文的完整复现。
15. 没有评测的数据, 不说“显著提升”。